

Resource Request algorithm

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{  
    int n, m;
```

```
    int allocation[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
```

```
    int available[MAX];
```

```
    int request[MAX];
```

```
    int i, j, k, process;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types:");
```

```
    scanf("%d", &m);
```

```
    printf("\nEnter allocation Matrix:\n");
```

```
    for(i=0; i<n; i++)
```

```
    {  
        for(j=0; j<m; j++)
```

```
        {  
            scanf("%d", &allocation[i][j]);
```

```
        }  
    }  
    printf("\nEnter Max Matrix:\n");
```

```
    for(i=0; i<n; i++)
```

```
    {  
        for(j=0; j<m; j++)
```

```
        {  
            scanf("%d", &max[i][j]);
```

```
        }  
    }  
    printf("\nEnter available resources:\n");
```

```
    for(j=0; j<m; j++)
```

```
    {  
        scanf("%d", &available[j]);
```

```
    }  
    for(i=0; i<n; i++)
```

```
    {  
        for(j=0; j<m; j++)
```

```
        {  
            need[i][j] = max[i][j] - allocation[i][j];
```

```
printf("Enter process numbers making request (0 to 100)\n", &n-1);
scanf("%d", &process);
```

```
printf("Enter Request vector:\n");
for (j=0; j<m; j++)
    scanf("%d", &request[j]);
```

```
for (j=0; j<m; j++) {
    if (request[j] > need[process][j]) {
        printf("Error: Request exceeds process need\n");
        return 0;
    }
}
```

```
for (j=0; j<m; j++) {
    if (request[j] > available[j]) {
        printf("Resource not available. Please wait.\n");
        return 0;
    }
}
```

```
for (j=0; j<m; j++) {
    available[j] -= request[j];
    allocation[process][j] += request[j];
    need[process][j] -= request[j];
}
```

```
int work[MAX], finish[MAX] = {0}, safeSeq[MAX];
int count = 0;
```

```
for (j=0; j<m; j++)
    work[j] = available[j];
```



```
while(count < n){
    int found = 0;
```

```
    for(i=0; i < n; i++){
        if(finish[i] == 0){
            int canExecute = 1;
```

```
            for(j=0; j < m; j++){
                if(need[i][j] > work[j]){
                    canExecute = 0;
                    break;
                }
            }
            if(canExecute){
                for(k=0; k < m; k++){
                    work[k] += allocation[i][k];
                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
    }
    if(!found) break;
}
if(count == n){
    printf("In Request can be granted. Safe Sequence!");
    for(i=0; i < n; i++){
        printf("%d ", safeSeq[i]);
        printf("\n");
    }
}
else {
    printf("In Request cannot be granted. Rolling back!\n");
}
```

```
for(j=0; j<m; j++){
    available[j] += request[j];
    allocation[process][j] = request[j];
    need[process][j] = request[j];
}
```

```
}
return 0;
```

```
}
```

output:

Enter number of processes: 5

Enter number of resource types: 3

Enter allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Max Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

Enter process numbers making request (0 to 4): 1

Enter request vector:

1 0 2

Request can be granted.

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2	2 3 0	1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1